

Bitácora de Mejoras y Cambios — Avance 3

Proyecto: Sistema de Monitoreo de Huella de Carbono (SIMO CO2)
Ingeniería Civil en Informática · Universidad de Los Lagos · TPA 2026

Esta bitácora documenta las mejoras realizadas entre el Avance 2 y el Avance 3 del proyecto SIMO CO2. El objetivo principal fue mejorar la arquitectura interna del sistema, aplicar patrones de diseño, ordenar la estructura del código y preparar una base escalable para futuras funcionalidades.

1. Comparativa General entre Avance 2 y Avance 3

Aspecto	Avance 2	Avance 3
Arquitectura	Código más básico y menos modular	Arquitectura modular y organizada
Patrones de Diseño	Uso limitado	Implementación de Factory, Observer y Singleton
Dashboard	No existía pantalla dedicada	Se añadió Dashboard.html
Gestión de Roles	Sin estructura de roles	Se agregaron Admin, Operador y Usuario
Creación de Objetos	Instancias manuales	Creación centralizada mediante factories
Escalabilidad	Difícil de extender	Preparado para nuevas funcionalidades

2. Cambios Técnicos Implementados

Patrón Factory

Se incorporaron las clases `SensorFactory.ts` y `WidgetFactory.ts` para centralizar la creación de sensores y widgets.

Patrón Observer

El módulo `DataDistributor.ts` fue reorganizado para manejar la distribución de datos hacia los widgets.

Patrón Singleton

El Repositorio ahora utiliza Singleton para mantener un único punto global de acceso.

Gestión de Roles

Se añadieron las clases Admin.ts, Operador.ts y Usuario.ts para separar responsabilidades.

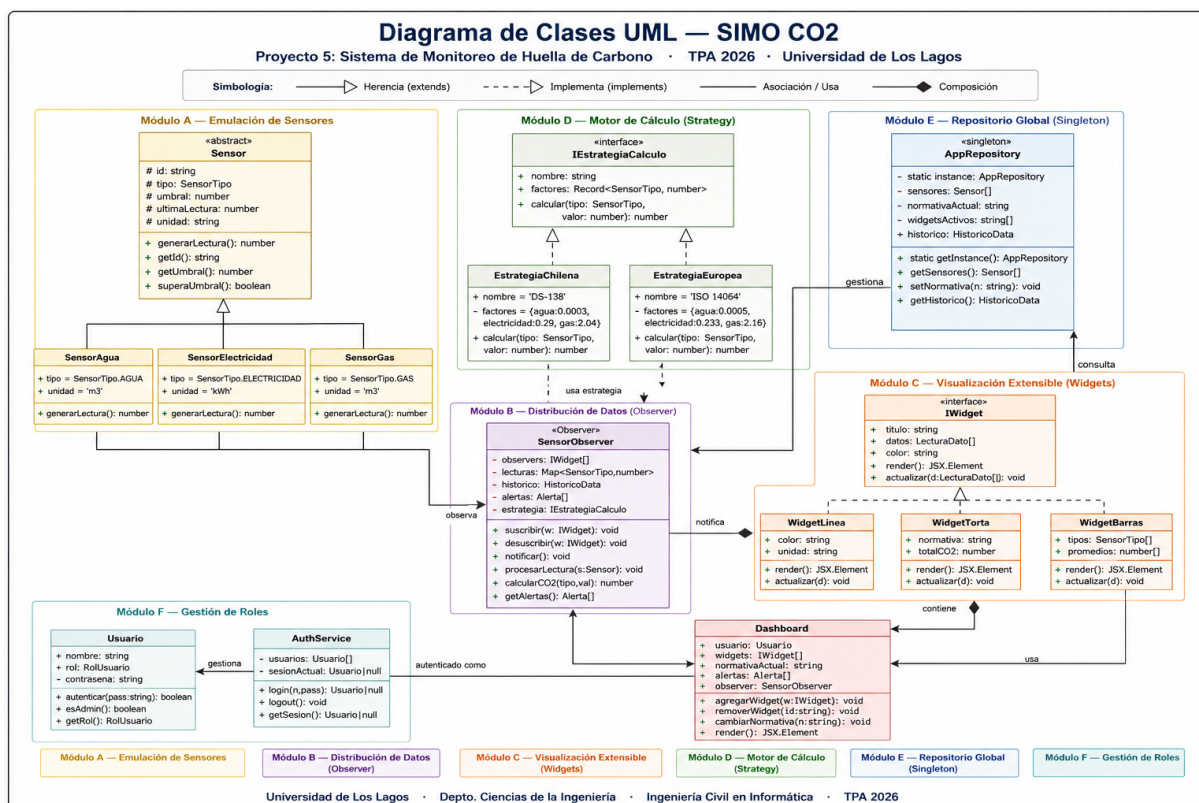
Dashboard

Se creó Dashboard.html como nueva interfaz principal para centralizar la visualización de datos.

3. Justificación de Patrones de Diseño

Los patrones de diseño fueron implementados para mejorar la mantenibilidad, reutilización y escalabilidad del sistema. El patrón **Factory** desacopla la creación de objetos. El patrón **Observer** facilita la actualización dinámica de widgets. El patrón **Singleton** asegura un único repositorio centralizado.

4. Diagrama UML Profesional



El diagrama UML fue reorganizado para representar claramente los módulos del sistema, las relaciones entre sensores, widgets y repositorios.

5. Mejoras Visuales y de Organización

Separación de carpetas por responsabilidades. Organización modular del código TypeScript. Mayor legibilidad en interfaces y clases. Preparación para una interfaz visual más dinámica. Mejor estructura para futuras integraciones frontend/backend.